

LABORATORY RECORD

PYTHON AUTOMATION

Selenium Automation Framework — Page Object Model, Data-Driven & PyTest Test Automation

Submitted by

Aryak Pal

University Roll No.: 12023052020025

B.Tech, Computer Science & Engineering (AI)

INSTITUTE OF ENGINEERING AND MANAGEMENT

Department of Computer Science & Engineering (AI)

Kolkata, West Bengal

Project Technical Report

Project: Selenium Automation Framework Development

Application Under Test: AutomationExercise — <https://automationexercise.com>

Framework: Selenium WebDriver with Python, pytest and Page Object Model

Individual Variation: CSV Parametrization + CSV Validation

Abstract

This report documents the complete Selenium Python automation framework developed for the Wipro capstone project. The framework separates page-specific locators and actions from test logic using the Page Object Model (POM), reads test data from CSV files, executes parameterized scenarios with pytest, validates the CSV inputs, captures screenshots when a test fails, manages browser creation through a reusable WebDriver factory, and generates an HTML execution report. A small Python unittest component is also included.

Project Objectives

- Build a maintainable Selenium automation framework in Python.
- Implement Page Object Model to separate UI interaction logic from test cases.
- Use CSV files as external test data and execute tests through pytest parametrization.
- Validate CSV structure and required fields before relying on test data.
- Provide reusable utilities for configuration, driver creation, CSV handling and screenshots.
- Capture screenshots automatically when a test fails.
- Generate a self-contained HTML report showing execution results.
- Include a small unittest-based component alongside pytest.

Contents

No.	Section
1	Framework Overview
2	Technology Stack and Tools
3	Project Directory Structure
4	Configuration Management
5	Page Object Model
6	CSV Data Handling and Parametrization
7	Test Cases
8	Reusable Utilities
9	PyTest Fixtures and Failure Screenshots
10	HTML Reporting
11	Environment Setup and Execution
12	Execution Flow
13	Test Execution Results
14	Maintainability and Repository Hygiene
15	Project Explanation Video
16	Conclusion

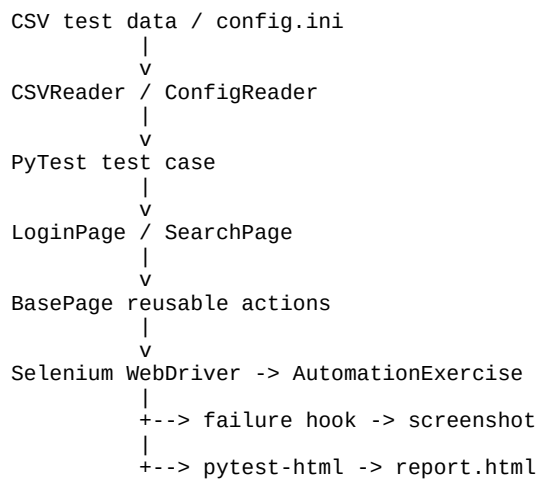
1. Framework Overview

The project is a Python-based Selenium automation framework for testing the AutomationExercise web application. It follows a layered design: test cases describe scenarios, page classes contain page-specific locators and actions, and utility modules provide common services such as configuration loading, CSV handling, driver creation and screenshots.

Core design principles

- Separation of concerns — UI details stay inside page classes.
- Reusability — BasePage and utility classes provide common behavior once.
- Data-driven testing — multiple scenarios are supplied through CSV rows.
- Maintainability — configuration and test data remain outside test logic.
- Observability — HTML reporting and failure screenshots provide execution evidence.

Framework flow



2. Technology Stack and Tools

Component	Used in the project
Programming language	Python
Browser automation	Selenium WebDriver
Test framework	PyTest
Unit-test component	Python unittest
Design pattern	Page Object Model (POM)
Test data	CSV files
Configuration	config.ini + ConfigReader
Browser	Google Chrome
Reporting	pytest-html
Environment	Python virtual environment (venv)

Selenium provides browser interaction, PyTest provides fixtures and parametrization, POM separates UI logic, CSV files provide external test data, and pytest-html provides the browser-viewable execution report.

3. Project Directory Structure

```
Selenium_Automation_Project/  
  config/  
  config.ini  
  pages/  
  base_page.py  
  login_page.py  
  search_page.py  
  reports/  
  screenshots/  
  report.html  
  test_data/  
  login_data.csv  
  search_data.csv  
  tests/  
  test_csv_unittest.py  
  test_csv_validation.py  
  test_login.py  
  test_search.py  
  utilities/  
  config_reader.py  
  csv_reader.py  
  driver_factory.py  
  screenshot.py  
  Project_Explanation/  
  README.md  
  conftest.py  
  requirements.txt  
  README.md  
  .gitignore
```

Folder responsibilities

config/ — Stores the application base URL.

pages/ — Contains BasePage and application-specific Page Objects.

reports/ — Stores report.html and failure screenshots.

test_data/ — Contains external CSV test data.

tests/ — Contains PyTest and unittest modules.

utilities/ — Reusable configuration, CSV, driver and screenshot helpers.

Project_Explanation/ — Contains the README with the Google Drive link to the recorded explanation video.

conftest.py — Provides the driver fixture and failure-report hook.

4. Configuration Management

The application URL is externalized so tests do not hardcode the environment value in every module.

config/config.ini

```
[DEFAULT]
base_url = https://automationexercise.com
```

ConfigReader reads the configuration using Python's configparser and returns config["DEFAULT"]["base_url"].

This makes the framework easier to maintain because a URL change can be made in one configuration file.

5. Page Object Model

The project uses BasePage, LoginPage and SearchPage. BasePage contains reusable Selenium operations; the other classes represent application-specific pages.

BasePage

```
class BasePage:
    def __init__(self, driver: WebDriver):
        self.driver = driver
        self.wait = WebDriverWait(driver, 10)
```

BasePage provides wait_for_element(), click(), enter_text() and get_text(). Explicit waits use Selenium expected conditions rather than relying on arbitrary sleep calls.

LoginPage

LoginPage contains the email, password, login button, sign-up/login and login-error locators. Its methods include open_login_page(), login() and get_login_error().

SearchPage

```
PRODUCTS_BUTTON = (By.CSS_SELECTOR, "a[href='/products']")
SEARCH_INPUT = (By.ID, "search_product")
SEARCH_BUTTON = (By.ID, "submit_search")
SEARCHED_PRODUCTS = (
    By.XPATH,
    "///div[@class='features_items']//div[contains(@class, 'productinfo')]"
)
```

SearchPage also contains handling for a possible closeable advertisement before interacting with the Products page.

6. CSV Data Handling and Parametrization

The individual project variation is CSV parametrization and CSV validation. Login and search scenarios are maintained as external data.

login_data.csv

```
email,password,expected_result
test@example.com,wrongpassword,failure
wrong@example.com,wrongpassword,failure
```

search_data.csv

```
product_name,expected_result
top,found
tshirt,found
dress,found
```

CSVReader

`read_csv()` uses `csv.DictReader` and returns a list of dictionaries. `validate_csv()` checks required columns and required fields. This keeps data validation reusable and independent from individual test cases.

PyTest parametrization

```
login_data = CSVReader.read_csv("test_data/login_data.csv")

@pytest.mark.parametrize("data", login_data)
def test_invalid_login(driver, data):
    ...
```

PyTest expands one test function into an execution for each CSV row, so adding another data row does not require duplicating the test logic.

7. Test Cases

Test module	Purpose	Executions
test_csv_unittest.py	Checks login and search CSV files contain data using unittest.	2
test_csv_validation.py	Validates required columns and non-empty fields.	2
test_login.py	Runs invalid-login scenarios from login_data.csv.	2
test_search.py	Runs product-search scenarios from search_data.csv.	3

Login scenarios

Each CSV row supplies an email and password. The test opens the configured application, navigates to the login page, submits the credentials and verifies the expected error message.

Search scenarios

Each CSV row supplies a product term. The test opens Products, performs the search and verifies that at least one matching product element is returned.

CSV validation and unittest

Two PyTest validation tests verify the required CSV structure. Two standard unittest methods verify that the CSV readers return data.

8. Reusable Utilities

Utility	Responsibility
ConfigReader	Reads base_url from config.ini.
CSVReader	Reads CSV data and validates columns/fields.
DriverFactory	Creates Chrome WebDriver with eager page-load strategy and maximizes the window.
Screenshot	Creates reports/screenshots and saves screenshots for failures.

DriverFactory

```
options = webdriver.ChromeOptions()
options.page_load_strategy = "eager"

driver = webdriver.Chrome(options=options)
driver.maximize_window()
```

Screenshot utility stores evidence under reports/screenshots and creates the directory when necessary.

9. PyTest Fixtures and Failure Screenshots

conftest.py provides a reusable driver fixture that creates the browser before a test and calls driver.quit() after the test completes.

```
@pytest.fixture
def driver():
    driver = DriverFactory.create_driver()
    yield driver
    driver.quit()
```

The same file implements pytest_runtest_makereport. When the call phase fails, the hook retrieves the driver from the test function and calls Screenshot.take_screenshot().

```
if report.when == "call" and report.failed:
    driver = item.funcargs.get("driver")
    Screenshot.take_screenshot(driver, item.name)
```

This gives automatic failure evidence without adding screenshot statements to every test.

10. HTML Reporting

The framework uses pytest-html to generate a self-contained HTML report.

Command used

```
pytest -v --html=reports/report.html --self-contained-html
```

The final execution report showed 9 tests, 0 failed, 9 passed and 0 skipped. The report can be opened directly in a browser to view the environment, summary, individual test results and durations.

Why some tests show 0 ms

The four CSV-only tests do not launch Chrome. They only read or validate small files, so their measured duration can round to 0 ms. The five Selenium UI tests take longer because they launch the browser and interact with the application.

11. Environment Setup and Execution

The project uses a virtual environment locally, while requirements.txt documents the dependencies needed to recreate it. The virtual environment itself is excluded from Git.

Setup

```
cd ~/Wipro_Capstone/Wipro_Selenium_Submission/Projects/Selenium_Automation_Project  
  
python3 -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt
```

Run tests

```
pytest -v
```

Generate report

```
pytest -v --html=reports/report.html --self-contained-html
```

The .gitignore excludes venv, Python cache directories, PyTest cache and macOS .DS_Store files.

12. Execution Flow

1	PyTest discovers test modules under tests/.
2	The driver fixture creates Chrome WebDriver.
3	CSV data is loaded and parameterized cases are expanded.
4	ConfigReader supplies the base URL.
5	Page Objects perform browser actions through BasePage.
6	Assertions verify the expected application behavior.
7	The fixture closes the browser.
8	If a test fails, the hook captures a screenshot.
9	pytest-html records the final result in report.html.

This flow keeps browser creation, page interaction, data handling, reporting and failure evidence in separate reusable components.

13. Test Execution Results

The completed framework was executed successfully after recreating the environment from requirements.txt. The final full suite produced nine passing tests.

Category	Count	Result
Unittest CSV checks	2	2 passed
CSV validation	2	2 passed
Parameterized login	2	2 passed
Parameterized search	3	3 passed
Total	9	9 passed / 0 failed

Verified report summary

The generated HTML report displays 0 failed, 9 passed and 0 skipped. It also lists the individual parameterized login and search executions. The CSV-only tests complete almost instantly, while browser-based tests show longer durations.

14. Maintainability and Repository Hygiene

The framework is designed so that future changes can be made with limited impact on unrelated components.

- Locators are centralized in Page Objects.
- Common browser actions are centralized in BasePage.
- External data is maintained in CSV files.
- The application URL is maintained in config.ini.
- Driver lifecycle is handled by the PyTest fixture.
- Failure screenshots are captured centrally through the hook.
- HTML reports are regenerated consistently after execution.

Repository hygiene

The local virtual environment and temporary caches are not required in GitHub. requirements.txt provides reproducibility, while .gitignore keeps generated and machine-specific files out of the repository. The project explanation video is stored on Google Drive because the recorded .mov file is approximately 280 MB.

15. Project Explanation Video

A separate Project_Explanation folder was added inside the Selenium_Automation_Project directory. It contains README.md with the Google Drive link to the recorded project explanation video.

```
Selenium_Automation_Project/  
■■■ Project_Explanation/  
■■■ README.md
```

Video link

https://drive.google.com/file/d/11zje2X04kJhbqyOMBlEf19z4pwsoKQaj/view?usp=drive_link

The video demonstrates the project structure, framework components, test data, utilities, test execution and HTML reporting. The video itself remains on Google Drive rather than being committed to GitHub because it is approximately 280 MB.

16. Conclusion

The completed project implements a structured Selenium automation framework using Python, PyTest and the Page Object Model. It separates configuration, page interactions, test data, utilities and test cases into dedicated components. CSV parametrization enables multiple login and search scenarios to execute through reusable test functions, while CSV validation checks the external data source.

The framework also includes a reusable WebDriver factory, explicit waits, automatic failure screenshots, a PyTest fixture for browser lifecycle management, a unittest component and an HTML execution report. The final verified execution produced **9 passed and 0 failed tests**.

The resulting structure provides a maintainable foundation that can be extended with additional Page Objects, test data and test modules without redesigning the core framework.

Submitted by

Aryak Pal

University Roll No.: 12023052020025

B.Tech, Computer Science & Engineering (AI)

INSTITUTE OF ENGINEERING AND MANAGEMENT

Department of Computer Science & Engineering (AI)

Kolkata, West Bengal